

Adding User Accounts

To add a new user account, you can run either of the following two commands as root.

```
# adduser [new_account]
# useradd [new_account]
```

When a new user account is added to the system, the following operations are performed.

1. His/her home directory is created (**/home/username** by default).
2. The following hidden files are copied into the user's home directory, and will be used to provide environment variables for his/her user session.

```
.bash_logout
.bash_profile
.bashrc
```

3. A mail spool is created for the user at **/var/spool/mail/username**.
4. A group is created and given the same name as the new user account.

Understanding /etc/passwd

The full account information is stored in the **/etc/passwd** file. This file contains a record per system user account and has the following format (fields are delimited by a colon).

```
[username]:[x]:[UID]:[GID]:[Comment]:[Home directory]:[Default shell]
```

1. Fields **[username]** and **[Comment]** are self explanatory.
2. The **x** in the second field indicates that the account is protected by a shadowed password (in **/etc/shadow**), which is needed to logon as **[username]**.
3. The **[UID]** and **[GID]** fields are integers that represent the User IDentification and the primary Group IDentification to which **[username]** belongs, respectively.
4. The **[Home directory]** indicates the absolute path to **[username]**'s home directory, and
5. The **[Default shell]** is the shell that will be made available to this user when he or she logs the system.

Understanding /etc/group

Group information is stored in the **/etc/group** file. Each record has the following format.

```
[Group name]:[Group password]:[GID]:[Group members]
```

1. **[Group name]** is the name of group.
2. An **x** in **[Group password]** indicates group passwords are not being used.
3. **[GID]**: same as in **/etc/passwd**.
4. **[Group members]**: a comma separated list of users who are members of **[Group name]**.

```
[gacanepa@dev1 ~]$ grep gacanepa /etc/passwd
gacanepa:x:1000:1000:Gabriel Cánepa:/home/gacanepa:/bin/bash
[gacanepa@dev1 ~]$ grep gacanepa /etc/group
gacanepa:x:1000:gacanepa
[gacanepa@dev1 ~]$
```

<http://www.tecmint.com>

After adding an account, you can edit the following information (to name a few fields) using the **usermod** command, whose basic syntax of usermod is as follows.

```
# usermod [options] [username]
```

Setting the expiry date for an account

Use the **--expiredate** flag followed by a date in **YYYY-MM-DD** format.

```
# usermod --expiredate 2014-10-30 tecmint
```

Adding the user to supplementary groups

Use the combined **-aG**, or **--append --groups** options, followed by a comma separated list of groups.

```
# usermod --append --groups root,users tecmint
```

Changing the default location of the user's home directory

Use the **-d**, or **--home** options, followed by the absolute path to the new home directory.

```
# usermod --home /tmp tecmint
```

Changing the shell the user will use by default

Use **--shell**, followed by the path to the new shell.

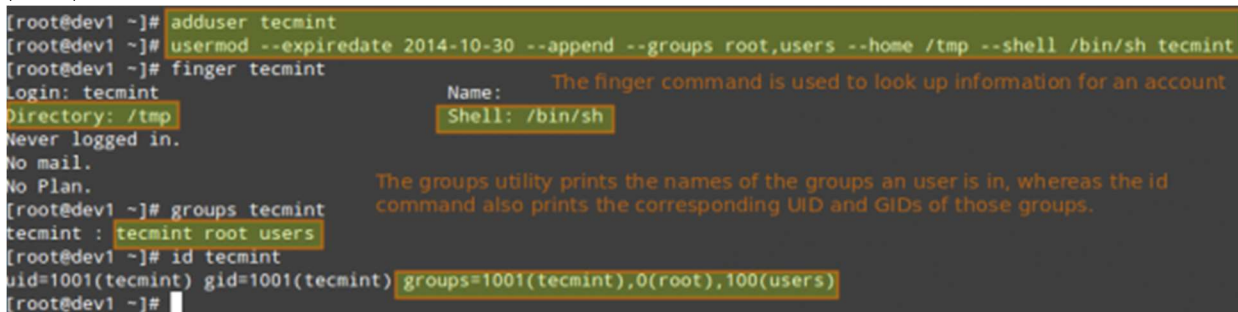
```
# usermod --shell /bin/sh tecmint
```

Displaying the groups an user is a member of

```
# groups tecmint
# id tecmint
```

Now let's execute all the above commands in one go.

```
# usermod --expiredate 2014-10-30 --append --groups root,users --home /tmp --shell /bin/sh tecmint
```



The screenshot shows a terminal session on a system with root access. The user first runs `adduser tecmint`. Then, they run `usermod --expiredate 2014-10-30 --append --groups root,users --home /tmp --shell /bin/sh tecmint`. To verify the changes, they run `finger tecmint`, which displays the user's details including the new home directory and shell. Next, they run `groups tecmint`, which lists the groups the user belongs to. Finally, they run `id tecmint`, which shows the user's UID, GID, and the groups they are a member of.

```
[root@dev1 ~]# adduser tecmint
[root@dev1 ~]# usermod --expiredate 2014-10-30 --append --groups root,users --home /tmp --shell /bin/sh tecmint
[root@dev1 ~]# finger tecmint
Login: tecmint                Name:
Directory: /tmp              Shell: /bin/sh
Never logged in.
No mail.
No Plan.
[root@dev1 ~]# groups tecmint
tecmint : tecmint root users
[root@dev1 ~]# id tecmint
uid=1001(tecmint) gid=1001(tecmint) groups=1001(tecmint),0(root),100(users)
```

In the example above, we will set the expiry date of the **tecmint** user account to **October 30th, 2014**. We will also add the account to the **root** and **users** group. Finally, we will set **sh** as its default shell and change the location of the home directory to **/tmp**:

Read Also:

1. [15 useradd Command Examples in Linux](#)

2. [15 usermod Command Examples in Linux](#)

For existing accounts, we can also do the following.

Disabling account by locking password

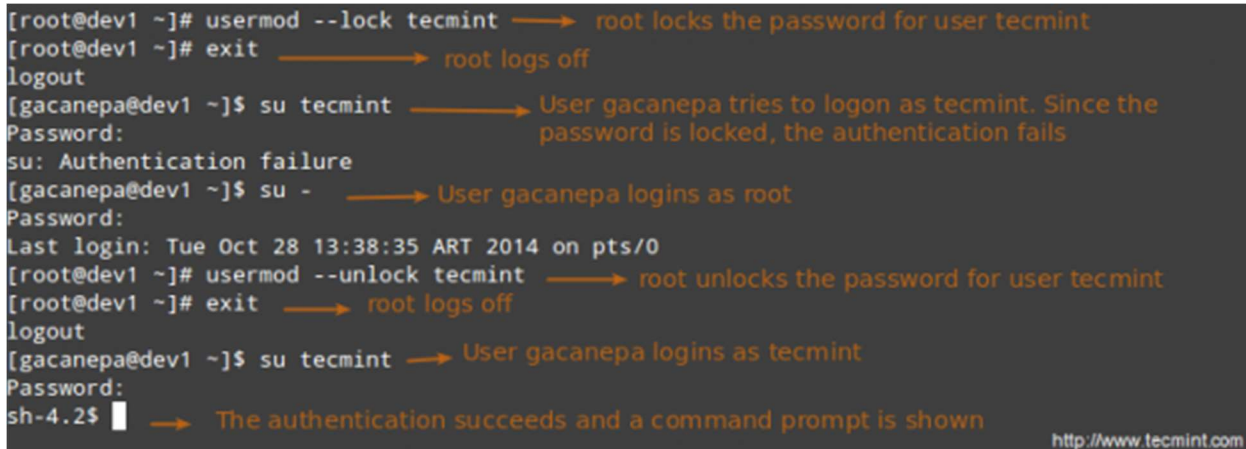
Use the **-L** (uppercase L) or the **-lock** option to lock a user's password.

```
# usermod --lock tecmint
```

Unlocking user password

Use the **-u** or the **-unlock** option to unlock a user's password that was previously blocked.

```
# usermod --unlock tecmint
```



The screenshot shows a terminal session where the user 'tecmint' is locked and then unlocked. The commands and their outputs are as follows:

```
[root@dev1 ~]# usermod --lock tecmint → root locks the password for user tecmint
[root@dev1 ~]# exit → root logs off
logout
[gacanepa@dev1 ~]$ su tecmint → User gacanepa tries to logon as tecmint. Since the
Password:                          password is locked, the authentication fails
su: Authentication failure
[gacanepa@dev1 ~]$ su - → User gacanepa logs in as root
Password:
Last login: Tue Oct 28 13:38:35 ART 2014 on pts/0
[root@dev1 ~]# usermod --unlock tecmint → root unlocks the password for user tecmint
[root@dev1 ~]# exit → root logs off
logout
[gacanepa@dev1 ~]$ su tecmint → User gacanepa logs in as tecmint
Password:
sh-4.2$ → The authentication succeeds and a command prompt is shown
```

http://www.tecmint.com

Creating a new group for read and write access to files that need to be accessed by several users

Run the following series of commands to achieve the goal.

```
# groupadd common_group # Add a new group
# chown :common_group common.txt # Change the group owner of common.txt to common_group
# usermod -aG common_group user1 # Add user1 to common_group
# usermod -aG common_group user2 # Add user2 to common_group
# usermod -aG common_group user3 # Add user3 to common_group
```

Deleting a group

You can delete a group with the following command.

```
# groupdel [group_name]
```

If there are files owned by **group_name**, they will not be deleted, but the group owner will be set to the **GID** of the group that was deleted.

Linux File Permissions

Besides the basic read, write, and execute permissions that we discussed in [Archiving Tools and Setting File Attributes](#) this series, there are other less used (but not less important) permission settings, sometimes referred to as “**special permissions**”.

Like the basic permissions discussed earlier, they are set using an octal file or through a letter (symbolic notation) that indicates the type of permission.

Deleting user accounts

You can delete an account (along with its home directory, if it's owned by the user, and all the files residing therein, and also the mail spool) using the **userdel** command with the **-remove** option.

```
# userdel --remove [username]
```

Group Management

Every time a new user account is added to the system, a group with the same name is created with the username as its only member. Other users can be added to the group later. One of the purposes of groups is to implement a simple access control to files and other system resources by setting the right permissions on those resources.

For example, suppose you have the following users.

1. user1 (primary group: user1)
2. user2 (primary group: user2)
3. user3 (primary group: user3)

All of them need **read** and **write** access to a file called **common.txt** located somewhere on your local system, or maybe on a network share that **user1** has created. You may be tempted to do something like,

```
# chmod 660 common.txt
OR
# chmod u=rw,g=rw,o= common.txt [notice the space between the last equal sign and the file name]
```

However, this will only provide **read** and **write** access to the owner of the file and to those users who are members of the group owner of the file (**user1** in this case). Again, you may be tempted to add **user2** and **user3** to group **user1**, but that will also give them access to the rest of the files owned by user **user1** and group **user1**.

This is where groups come in handy, and here's what you should do in a case like this.

Understanding Setuid

When the **setuid** permission is applied to an executable file, an user running the program inherits the effective privileges of the program's owner. Since this approach can reasonably raise security concerns, the number of files with setuid permission must be kept to a minimum. You will likely find programs with this permission set when a system user needs to access a file owned by root.

Summing up, it isn't just that the user can execute the binary file, but also that he can do so with root's privileges. For example, let's check the permissions of **/bin/passwd**. This binary is used to change the password of an account, and modifies the **/etc/shadow** file. The superuser can change anyone's password, but all other users should only be able to change their own.

```
[root@dev1 ~]# ls -l /bin/passwd
-rwsr-xr-x. 1 root root 27832 Jun 10 03:27 /bin/passwd
[root@dev1 ~]#
```

→ This s stands for setuid

<http://www.tecmint.com>

Thus, any user should have permission to run **/bin/passwd**, but only root will be able to specify an account. Other users can only change their corresponding passwords.

```
[gacanepa@dev1 ~]$ passwd tecmint
passwd: Only root can specify a user name.
[gacanepa@dev1 ~]$ passwd
Changing password for user gacanepa.
Changing password for gacanepa.
(current) UNIX password:
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[gacanepa@dev1 ~]$
```

<http://www.tecmint.com>

Understanding Setgid

When the **setgid** bit is set, the effective **GID** of the real user becomes that of the group owner. Thus, any user can access a file under the privileges granted to the group owner of such file. In addition, when the setgid bit is set on a directory, newly created files inherit the same group as the directory, and newly created subdirectories will also inherit the setgid bit of the parent directory. You will most likely use this approach whenever members of a certain group need access to all the files in a directory, regardless of the file owner's primary group.

```
# chmod g+s [filename]
```

To set the **setgid** in octal form, prepend the number **2** to the current (or desired) basic permissions.

```
# chmod 2755 [directory]
```

Setting the SETGID in a directory

```
[root@dev1 ~]# ls -l
total 0
drwxr-xr-x. 3 root root 21 Oct 29 22:47 backups
[root@dev1 ~]# chmod g+s backups
[root@dev1 ~]# ls -l
total 0
drwxr-sr-x. 3 root root 21 Oct 29 22:47 backups
[root@dev1 ~]# mkdir backups/testdir
[root@dev1 ~]# ls -ld backups/testdir
drwxr-sr-x. 2 root root 6 Oct 29 22:48 backups/testdir
[root@dev1 ~]#
```

The setgid is applied to a directory (the g stands for 'group' and the s stands for 'setgid'). In other words, the setgid is a permission that only applies to groups.

Newly created directories inherit the setgid bit from the parent directory.

<http://www.tecmint.com>

Understanding Sticky Bit

When the “**sticky bit**” is set on files, Linux just ignores it, whereas for directories it has the effect of preventing users from deleting or even renaming the files it contains unless the user owns the directory, the file, or is root.

```
# chmod o+t [directory]
```

To set the **sticky bit** in octal form, prepend the number **1** to the current (or desired) basic permissions.

```
# chmod 1755 [directory]
```

Without the sticky bit, anyone able to write to the directory can delete or rename files. For that reason, the sticky bit is commonly found on directories, such as **/tmp**, that are world-writable.

```
[root@dev1 ~]# ls -ld /tmp → This t indicates that the sticky bit is set for /tmp
drwxrwxrwt. 7 root root 108 Oct 30 08:59 /tmp
[root@dev1 ~]# exit
logout
[gacane@dev1 ~]$ touch /tmp/myfile
[gacane@dev1 ~]$ ls -lR /tmp
/tmp:
total 0
-rw-rw-r--. 1 gacane gacane 0 Oct 30 09:01 myfile
[gacane@dev1 ~]$ su tecmint
Password:
[tecmint@dev1 gacane]$ rm /tmp/myfile
rm: remove write-protected regular empty file '/tmp/myfile'? y
rm: cannot remove '/tmp/myfile': Operation not permitted
[tecmint@dev1 gacane]$
```

root logs out and user gacane creates an empty file within /tmp.
gacane logs out and user tecmint attempts to delete the file.
Since the sticky bit is set for the parent directory, the delete operation fails.

<http://www.tecmint.com>

Special Linux File Attributes

There are other attributes that enable further limits on the operations that are allowed on files. For example, prevent the file from being renamed, moved, deleted, or even modified. They are set with the [chattr command](#) and can be viewed using the **lsattr** tool, as follows.

```
# chattr +i file1
# chattr +a file2
```

After executing those two commands, **file1** will be immutable (which means it cannot be moved, renamed, modified or deleted) whereas **file2** will enter append-only mode (can only be open in append mode for writing).


```

[root@dev1 ~]# touch file1
[root@dev1 ~]# chattr +i file1
[root@dev1 ~]# lsattr file1
----i----- file1
[root@dev1 ~]# rm file1
rm: remove regular empty file 'file1'? y
rm: cannot remove 'file1': Operation not permitted
[root@dev1 ~]# chattr -i file1
[root@dev1 ~]# lsattr file1
----- file1
[root@dev1 ~]# rm file1
rm: remove regular empty file 'file1'? y
[root@dev1 ~]# echo "Hi there" > file2
[root@dev1 ~]# chattr +a file2
[root@dev1 ~]# cat /dev/null > file2
-bash: file2: Operation not permitted
[root@dev1 ~]# echo "This is another line" >> file2
[root@dev1 ~]# cat file2
Hi there
This is another line
[root@dev1 ~]# lsattr file2
-----a----- file2
[root@dev1 ~]# chattr -a file2
[root@dev1 ~]# cat /dev/null > file2
[root@dev1 ~]#

```

When the immutable attribute is set for a file, not even root can delete it!

If we need to modify a file that has the immutable attribute set, we will have to remove the attribute first.

You cannot delete the contents of a file that has the append-only attribute set. However, you can append content to it.

You need to remove the append-only attribute if you need to delete some of the contents of the file.

<http://www.tecmint.com>

Accessing the root Account and Using sudo

One of the ways users can gain access to the root account is by typing.

```
$ su
```

and then entering root's password.

If authentication succeeds, you will be logged on as **root** with the current working directory as the same as you were before. If you want to be placed in root's home directory instead, run.

```
$ su -
```

and then enter root's password.

```
[gacanepa@dev1 ~]$ pwd
/home/gacanepa
[gacanepa@dev1 ~]$ su
Password:
[root@dev1 gacanepa]# pwd
/home/gacanepa
[root@dev1 gacanepa]# exit
exit
[gacanepa@dev1 ~]$ su -
Password:
Last login:  on pts/0
[root@dev1 ~]# pwd
/root
[root@dev1 ~]#
```

<http://www.tecmint.com>

The above procedure requires that a normal user knows root's password, which poses a serious security risk. For that reason, the sysadmin can configure the **sudo** command to allow an ordinary user to execute commands as a different user (usually the superuser) in a very controlled and limited way. Thus, restrictions can be set on a user so as to enable him to run one or more specific privileged commands and no others.

Read Also: [Difference Between su and sudo User](#)

To authenticate using **sudo**, the user uses his/her own password. After entering the command, we will be prompted for our password (not the superuser's) and if the authentication succeeds (and if the user has been granted privileges to run the command), the specified command is carried out.

To grant access to sudo, the system administrator must edit the **/etc/sudoers** file. It is recommended that this file is edited using the **visudo** command instead of opening it directly with a text editor.

```
# visudo
```

This opens the **/etc/sudoers** file using **vim** (you can follow the instructions given in [Install and Use vim as Editor – Part 2](#) of this series to edit the file).

These are the most relevant lines.

```
Defaults    secure_path="/usr/sbin:/usr/bin:/sbin"
root        ALL=(ALL) ALL
tecmint      ALL=/bin/yum update
gacanepa     ALL=NOPASSWD:/bin/updatedb
%admin       ALL=(ALL) ALL
```

Let's take a closer look at them.

```
Defaults    secure_path="/usr/sbin:/usr/bin:/sbin:/usr/local/bin"
```

This line lets you specify the directories that will be used for **sudo**, and is used to prevent using user-specific directories, which can harm the system.

The next lines are used to specify permissions.

```
root        ALL=(ALL) ALL
```


1. The first **ALL** keyword indicates that this rule applies to all hosts.
2. The second **ALL** indicates that the user in the first column can run commands with the privileges of any user.
3. The third **ALL** means any command can be run.

```
tecmin      ALL=/bin/yum update
```

If no user is specified after the = sign, sudo assumes the root user. In this case, user **tecmin** will be able to run **yum update** as root.

```
gacanepa    ALL=NOPASSWD:/bin/updatedb
```

The **NOPASSWD** directive allows user gacanepa to run **/bin/updatedb** without needing to enter his password.

```
%admin      ALL=(ALL) ALL
```

The % sign indicates that this line applies to a group called “**admin**”. The meaning of the rest of the line is identical to that of a regular user. This means that members of the group “**admin**” can run all commands as any user on all hosts.

To see what privileges are granted to you by sudo, use the “**-l**” option to list them.

Before

```
[gacanepa@dev1 root]$ sudo -l
We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

[sudo] password for gacanepa:
Sorry, user gacanepa may not run sudo on dev1.
[gacanepa@dev1 root]$
```



Before and after adding user
gacanepa to the sudoers file

After

```
User gacanepa may run the following commands on this host:
(root) NOPASSWD: /bin/updatedb
http://www.tecmint.com
```